

SEMINARARBEIT

im Studiengang Informatik

Lehrveranstaltung Continuous Integration (S4-CONINT)

Secret Notes - CI/CD Semesterprojekt

Ausgeführt von: Abdelrahman Salama, Kyrillos Abdalla, Shez Abbas
Soltani, Mohamad Aldulemi

Personenkennzeichen: <Personenkennzeichen einfügen>

BegutachterIn: <BegutachterIn einfügen>

Wien, 07.07.2026

Kurzfassung

Diese Dokumentation beschreibt die Umsetzung des Semesterprojekts „Secret Notes“ im Rahmen der Lehrveranstaltung Continuous Integration. Ziel des Projekts war es, eine kleine Webanwendung zu entwickeln, mit der geheime Notizen erstellt und später nur mit der richtigen Passphrase wieder gelesen werden können. Die Anwendung besteht aus einem Frontend, einem Node.js/Fastify-Backend und einer PostgreSQL-Datenbank auf AWS RDS. In der Datenbank werden die Notizeninhalte nicht im Klartext gespeichert, sondern nur in verschlüsselter Form.

Neben der eigentlichen Anwendung lag der Schwerpunkt vor allem auf dem DevOps-Prozess. Frontend und Backend wurden als Docker-Images gebaut, in Docker Hub veröffentlicht und auf AWS EC2 mit Docker Compose betrieben. Für die Automatisierung wurden sowohl GitHub-Actions-Pipelines (cloud-hosted) als auch Jenkins Multibranch-Pipelines (self-hosted, ebenfalls auf AWS EC2) eingerichtet. Beide führen Codeanalyse mit SonarQube, Security-Scans mit Snyk, Unit-Tests, Docker-Builds, das Veröffentlichen der Images, das Deployment sowie E2E- und Performance-Tests mit Playwright und k6 aus. Zusätzlich wurde ein Blue/Green-Deployment umgesetzt, damit eine neue Version zuerst in einer separaten Umgebung getestet und danach live geschaltet werden kann.

Schlagwörter: Continuous Integration, Continuous Deployment, Docker, AWS, Blue/Green Deployment, SonarQube, Snyk, PostHog, Jenkins, PostgreSQL

Abstract

This documentation describes the implementation of the Secret Notes semester project for the Continuous Integration course. The application consists of a frontend, a Node.js/Fastify backend and a PostgreSQL database hosted on AWS RDS. Users can create secret notes and decrypt them only with the correct passphrase. The note content is stored only in encrypted form in the database.

The main focus is the DevOps and CI/CD process. Docker images for frontend and backend were built, pushed to Docker Hub and deployed on AWS EC2 using Docker Compose. The GitHub Actions pipelines include linting with SonarQube, security scans with Snyk, unit tests, Docker builds, Docker Hub delivery, deployment, Playwright E2E tests, k6 performance tests and a Blue/Green traffic switch.

Keywords: Continuous Integration, Continuous Deployment, Docker, AWS, Blue/Green Deployment, SonarQube, Snyk, PostHog, Jenkins, PostgreSQL

Inhaltsverzeichnis

Kurzfassung	2
Abstract	3
Inhaltsverzeichnis	4
1 Einleitung und Projektziel	6
Team und Aufgabenverteilung	6
2 Anforderungsübersicht	7
3 Anwendung und Architektur	8
4 Docker-Dateien und Containerisierung	9
5 Versionsverwaltung und Branching-Strategie	13
6 AWS-Infrastruktur	14
7 Docker Hub	18
8 CI/CD-Server: GitHub Actions und Jenkins	20
9 Pipeline-Stages	22
10 Tests und Testabdeckung	24
11 SonarQube	25
12 Snyk Security Scan	27
13 PostHog Feature Toggle und A/B-Test	28
14 Blue/Green Deployment	29
15 Fehlerbehandlung und Benachrichtigung	30
16 Benutzerfunktionen und User Stories	30
17 Setup- und Bedienungsanleitung	30
18 Probleme, Entscheidungen und Lösungen	31

19 Logging und Monitoring (Outlook).....	33
20 Fazit	34
21 Literaturverzeichnis	34
22 Abbildungsverzeichnis.....	34
23 Abkürzungsverzeichnis.....	34
24 Dokumentationstabelle KI-basierte Hilfsmittel.....	36

1 Einleitung und Projektziel

Secret Notes ist eine sichere Notiz-Anwendung, bei der Notizen nicht im Klartext, sondern verschlüsselt in einer relationalen Datenbank gespeichert werden. Beim Lesen einer Notiz muss der Benutzer die korrekte Passphrase angeben. Nur dann entschlüsselt das Backend den gespeicherten Inhalt und gibt ihn an das Frontend zurück.

Das technische Ziel des Projekts ist nicht nur die Anwendung selbst, sondern vor allem der Aufbau einer vollständigen CI/CD-Umgebung. Dazu gehören cloud-hosted CI mit GitHub Actions, self-hosted CI mit Jenkins, statische Analyse mit SonarQube, Security-Scanning mit Snyk, Feature Toggles mit PostHog, Docker Hub als Registry und ein Blue/Green Deployment auf AWS.

Team und Aufgabenverteilung

Das Projekt wurde von einem vierköpfigen Team umgesetzt. Die Aufgaben wurden bereichsweise aufgeteilt, wobei alle Mitglieder an Code Reviews, Bugfixing und der gemeinsamen Fehlersuche in der CI/CD-Pipeline beteiligt waren. Die folgende Tabelle zeigt die primäre Verantwortung je Bereich.

Bereich	Verantwortlich	Kurzbeschreibung
Backend (Fastify, Verschlüsselung, REST-API)	Kyrillos Abdalla	Umsetzung von Feature A (Notiz verschlüsselt speichern) und Feature B (Notiz nur mit korrekter Passphrase lesen), Backend-Unit-Tests.
Frontend (React-UI, PostHog-Integration)	Abdelrahman Salama	Umsetzung von Feature C (UI-Variante A/B via PostHog), Frontend-Komponenten und Component-/Unit-Tests.
AWS-Infrastruktur, GitHub Actions, SonarQube/Snyk	Shez Abbas Soltani	Einrichtung von EC2/RDS, Security Groups, Konfiguration der cloud-hosted GitHub-Actions-Pipelines sowie SonarQube- und Snyk-Integration.

Self-hosted CI (Jenkins), Blue/Green-Deployment	Mohamad Aldulemi	Aufbau der Jenkins Multibranch-Pipelines für Backend und Frontend (inkl. Node.js-, SonarScanner-, Docker- und SSH-Agent- Tooling im Jenkins- Container), Fehlersuche bei der self-hosted CI sowie Betrieb des Blue/Green- Deployments auf AWS.
--	------------------	---

2 Anforderungsübersicht

Die Projektangabe fordert zwei getrennte Pipelines für Frontend und Backend. Jede Pipeline soll mindestens Linting, Unit Testing, Build, Deliver und Deploy enthalten. Darüber hinaus werden E2E- und Performance-Tests auf der Staging-Umgebung ausgeführt. Die vollständige Pipeline läuft auf dem Branch deploy/production; auf main werden Linting, Testing und Build ausgeführt.

Bereich	Umsetzung im Projekt
Frontend	Weboberfläche zum Erstellen und Lesen von Secret Notes; Dockerized und über nginx ausgeliefert.
Backend	Node.js/Fastify REST-API zur Verschlüsselung, Speicherung, Entschlüsselung und Auslieferung von Notizen.
Datenbank	PostgreSQL auf AWS RDS; Speicherung von ciphertext, iv, salt und auth_tag.
CI/CD	GitHub Actions für cloud-hosted CI/CD und Jenkins als self-hosted CI-Server auf AWS EC2.
Qualität und Security	SonarQube für statische Analyse, Snyk für Dependency- und Container-Security.
Deployment	Docker Compose auf AWS EC2 mit Blue/Green-Ansatz und nginx-Router.
Feature Toggle	PostHog Feature Flag für UI-Variation / A/B-Test mit 50%-Rollout.

3 Anwendung und Architektur

Die Anwendung ist in drei Hauptkomponenten geteilt: Frontend, Backend und Datenbank. Das Frontend stellt Formulare zum Erstellen und Lesen von Notizen bereit. Das Backend nimmt REST-Anfragen entgegen, verschlüsselt neue Notizen serverseitig und speichert nur verschlüsselte Werte in PostgreSQL. Beim Lesen prüft das Backend die Passphrase und gibt nur bei korrekter Passphrase den Klartext zurück.

Bereich	Umsetzung im Projekt
Frontend	Plain HTML/JavaScript bzw. buildfähiges Frontend, ausgeliefert als statisches nginx-Image.
Backend	Fastify-Service mit REST-Endpunkten GET /api/notes, POST /api/notes und POST /api/notes/:id/reveal.
Verschlüsselung	Serverseitige Verschlüsselung mit passphrasebasiertem Schlüsselmaterial, Salt, IV und Auth Tag.
Kommunikation	nginx-Router leitet /api/* an das aktive Backend und alle anderen Pfade an das aktive Frontend weiter.
Datenhaltung	RDS PostgreSQL speichert keine Klartexte. Sichtbar sind nur technische Cipher-Spalten.

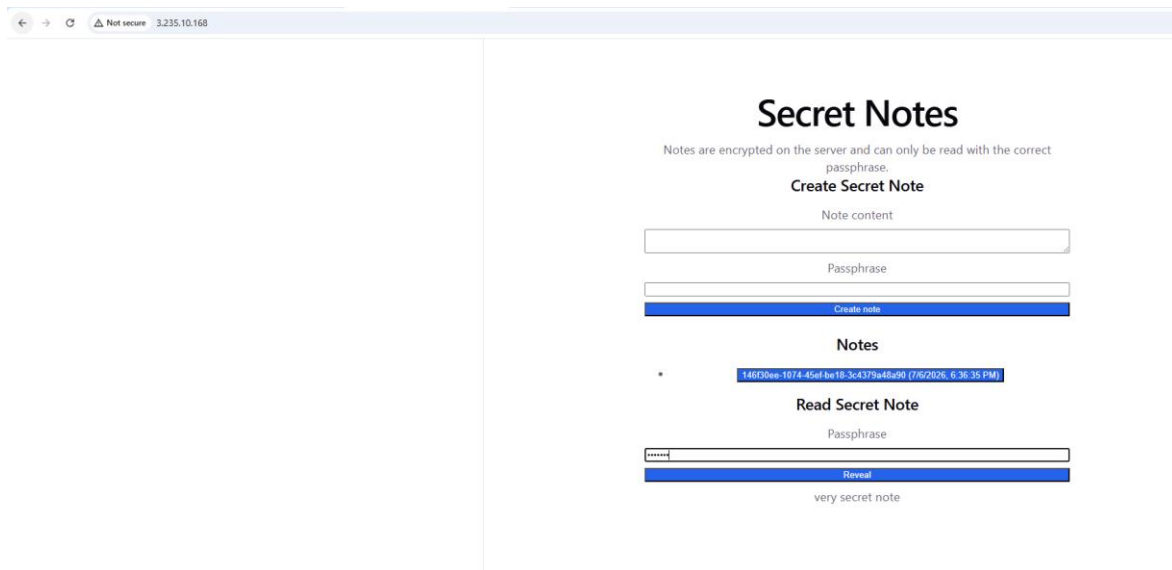


Abbildung 1: Secret Notes Frontend mit erstellter und korrekt entschlusselter Notiz.

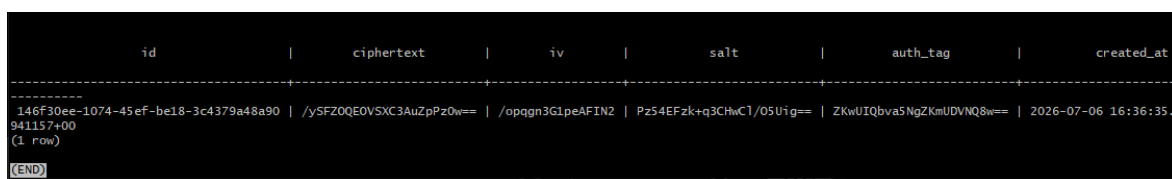


Abbildung 2: Datenbanknachweis: gespeicherte Notiz nur als ciphertext, iv, salt und auth_tag.

4 Docker-Dateien und Containerisierung

Frontend und Backend werden jeweils in ein eigenes Docker-Image verpackt. Das Backend-Image installiert die Node.js-Abhängigkeiten und startet den Fastify-Server. Das Frontend-Image baut die statischen Dateien und liefert diese über nginx aus. Beide Images werden in Docker Hub gespeichert und vom Deployment-Server per Docker Compose gezogen.

Bereich	Umsetzung im Projekt
Backend Dockerfile	Installiert dependencies, kopiert Backend-Code und startet den Node.js/Fastify-Service auf Port 3000.
Frontend Dockerfile	Baut das Frontend und liefert die statischen Dateien über nginx auf Port 80 aus.
docker-compose.blue-green.yml	Definiert backend-blue, backend-green, frontend-blue, frontend-green und router.
Environment	DATABASE_URL wird als Secret/Umgebungsvariable übergeben; Frontend nutzt /api als API-Basis.

Ports	Port 80 = Live-Traffic; Port 8081 = Green/Staging Preview; Backend intern auf Port 3000.
--------------	--



```

1 FROM node:22-alpine AS base
2 WORKDIR /app
3
4 COPY package*.json ./
5 RUN npm ci --omit=dev
6
7 COPY src ./src
8
9 EXPOSE 3000
10 CMD ["node", "src/server.js"]
11

```

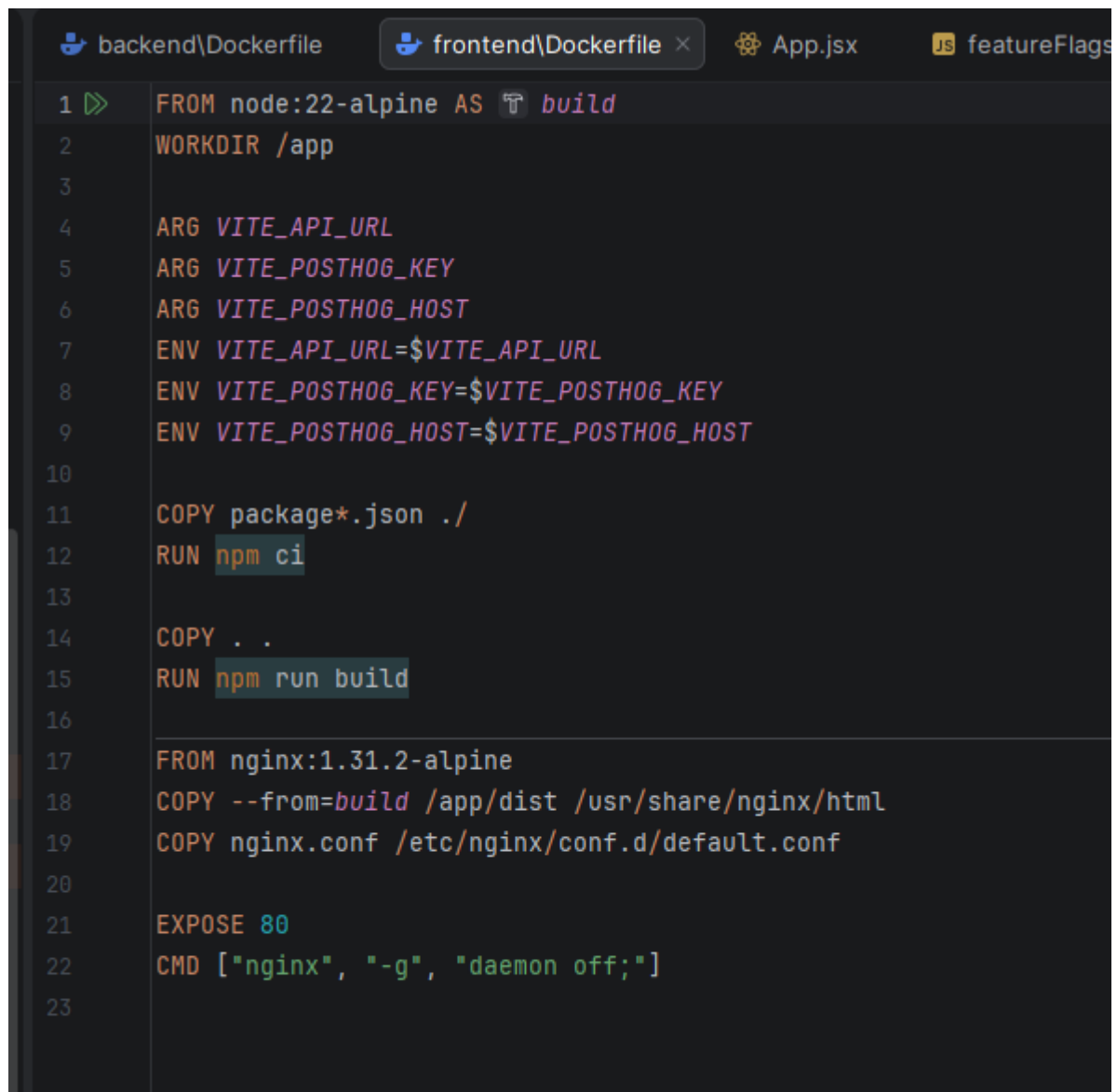
Abbildung 3: Backend Dockerfile - Node.js-Basis-Image, Installation der Dependencies und Start des Fastify-Servers.

```

1  # Runs on the AWS EC2 host. Two colors of each service run side by side;
2  # router.conf decides which color receives real user traffic.
3  services:
4    backend-blue:
5      image: ${DOCKERHUB_USERNAME}/secret-notes-backend:${BLUE_TAG:-latest}
6      environment:
7        DATABASE_URL: ${DATABASE_URL}
8        NODE_TLS_REJECT_UNAUTHORIZED: "0"
9      expose:
10       - "3000"
11
12   backend-green:
13     image: ${DOCKERHUB_USERNAME}/secret-notes-backend:${GREEN_TAG:-latest}
14     environment:
15       DATABASE_URL: ${DATABASE_URL}
16       NODE_TLS_REJECT_UNAUTHORIZED: "0"
17     expose:
18       - "3000"
19
20   frontend-blue:
21     image: ${DOCKERHUB_USERNAME}/secret-notes-frontend:${BLUE_TAG:-latest}
22     expose:
23       - "80"
24
25   frontend-green:
26     image: ${DOCKERHUB_USERNAME}/secret-notes-frontend:${GREEN_TAG:-latest}
27     expose:
28       - "80"
29   router:
30     image: nginx:1.27-alpine
31     volumes:
32       - ./router.conf:/etc/nginx/conf.d/default.conf:ro
33       - ./active-color.conf:/etc/nginx/active-color.conf:ro
34     ports:
35       - "80:80"
36       - "8081:8081" # staging (green) preview port, used by the E2E stage
37     depends_on:
38       - backend-blue
39       - backend-green
40       - frontend-blue
41       - frontend-green
42

```

Abbildung 4: docker-compose.blue-green.yml - Definition der Blue/Green-Services (Backend, Frontend, Router).



```
1 >> FROM node:22-alpine AS build
2 WORKDIR /app
3
4 ARG VITE_API_URL
5 ARG VITE_POSTHOG_KEY
6 ARG VITE_POSTHOG_HOST
7 ENV VITE_API_URL=$VITE_API_URL
8 ENV VITE_POSTHOG_KEY=$VITE_POSTHOG_KEY
9 ENV VITE_POSTHOG_HOST=$VITE_POSTHOG_HOST
10
11 COPY package*.json ./
12 RUN npm ci
13
14 COPY . .
15 RUN npm run build
16
17 FROM nginx:1.31.2-alpine
18 COPY --from=build /app/dist /usr/share/nginx/html
19 COPY nginx.conf /etc/nginx/conf.d/default.conf
20
21 EXPOSE 80
22 CMD ["nginx", "-g", "daemon off;"]
23
```

Abbildung 5: Frontend Dockerfile - Multi-Stage-Build mit Vite und Auslieferung ueber nginx.

5 Versionsverwaltung und Branching-Strategie

Das Projekt wird in GitHub versioniert. main dient zur laufenden Integration und startet Linting, Testing und Build. Der Branch deploy/production triggert die vollständige Delivery- und Deployment-Pipeline. Dadurch wird verhindert, dass jeder normale main-Push sofort produktionsnahe Deployments auslöst.

Bereich	Umsetzung im Projekt
main	Führt Linting, Unit Tests und Docker Build für Frontend und Backend aus.
deploy/production	Führt alle Stages inklusive DockerHub Push, Deploy, E2E/Performance und Blue/Green Switch aus.
Commit-Strategie	Kleine, nachvollziehbare Commits für CI/CD-Fixes, Dependency-Fixes, E2E-Fixes und Deployment-Konfiguration.
Secrets	Sensible Werte wie Tokens und SSH-Key liegen in GitHub Repository Secrets und werden nicht im Code gespeichert.

Name ↕	Last updated
AWIS_EC2_HOST	now  
AWIS_EC2_SSH_KEY	1 hour ago  
AWIS_EC2_USER	1 hour ago  
DOCKERHUB_TOKEN	19 hours ago  
DOCKERHUB_USERNAME	19 hours ago  
POSTHOG_FEATURE_FLAG	19 hours ago  
POSTHOG_HOST	19 hours ago  
POSTHOG_PUBLIC_KEY	19 hours ago  
SLACK_WEBHOOK_URL	19 hours ago  
SNYK_TOKEN	19 hours ago  
SONAR_HOST_URL	1 hour ago  
SONAR_PROJECT_KEY_BACKEND	19 hours ago  
SONAR_PROJECT_KEY_FRONTEND	19 hours ago  
SONAR_TOKEN	19 hours ago  
STAGING_API_URL	now  
STAGING_BLUE_URL	1 minute ago  
STAGING_GREEN_URL	now  

Abbildung 6: GitHub Repository Secrets fuer Deployment, SonarQube, Snyk, Slack und PostHog.

6 AWS-Infrastruktur

Die AWS-Infrastruktur besteht aus einer EC2-Instanz für die Anwendung, einer EC2-Instanz für SonarQube/Jenkins und einer RDS PostgreSQL-Datenbank. Die Anwendung wird per Docker Compose auf der App-EC2 betrieben. Die RDS-Datenbank ist nicht öffentlich erreichbar und wird aus der EC2-Umgebung angesprochen.

Bereich	Umsetzung im Projekt
App EC2	Hostet Docker, Docker Compose, nginx-Router sowie Blue/Green-Frontend- und Backend-Container.
Sonar/Jenkins EC2	Hostet SonarQube auf Port 9000 und Jenkins auf Port 8080.
RDS PostgreSQL	Instanz secret-notes-db, Engine PostgreSQL, Klasse db.t3.micro, Status Available, Region us-east-1 (AZ us-east-1c). Zugriff ausschliesslich über die Security Group des App-Servers (rds-ec2-1) sowie zusätzlich eine feste Wartungs-IP (85.127.92.38/32) - keine öffentliche Erreichbarkeit aus dem Internet. Engine-Version PostgreSQL 17.9, DB-Name secretnotes, Master-User postgres, Instance Class db.t3.micro (2 vCPU, 1 GB RAM), Storage 20 GiB General Purpose SSD (gp3, 3000 IOPS) mit Storage Autoscaling bis 1000 GiB, Verschlüsselung at rest aktiviert (AWS KMS), Multi-AZ deaktiviert.
Security Groups	App-Server (secret-notes-app-sg): Port 80, 3000, 8081 und 22 offen für 0.0.0.0/0. Sonar/Jenkins-Server (sonarqube-sg): Port 9000 (SonarQube) offen für 0.0.0.0/0, da GitHub Actions von wechselnden IPs aus scannt; Port 8080 (Jenkins) und Port 22 (SSH) sind bewusst auf eine einzelne feste Quell-IP (85.127.92.38/32) eingeschränkt, um Wartungszugriff und Dashboard nicht öffentlich zugaenglich zu machen.

Netzwerk	RDS ist über interne AWS-Verbindung erreichbar; App verwendet DATABASE_URL mit SSL-Parameter.
AWS Academy Lab / IP-Wechsel	Die Infrastruktur läuft in einem AWS Academy Learner Lab. Nach Ablauf bzw. Neustart der Lab-Session werden die EC2-Instanzen gestoppt und erhalten beim nächsten Start eine neue öffentliche IP-Adresse (keine Elastic IP reserviert). Nach jedem Neustart mussten die IP-abhängigen GitHub Secrets (AWS_EC2_HOST, STAGING_GREEN_URL, STAGING_BLUE_URL, SONAR_HOST_URL) sowie die entsprechenden Jenkins-Umgebungsvariablen manuell aktualisiert werden.

▼ Inbound rules				
Filter rules < 1 >				
Name	Security group rule ID	Port range	Protocol	Source
–	sgr-05922676e98742816	80	TCP	0.0.0.0/0
–	sgr-096139b3a2292fa52	3000	TCP	0.0.0.0/0
–	sgr-03898123cb614340e	22	TCP	0.0.0.0/0
–	sgr-0c4b8c037b9bdde8a	8081	TCP	0.0.0.0/0
–	sgr-09d5e9cd8173a45f4	8080	TCP	0.0.0.0/0
▼ Outbound rules				
Filter rules < 1 >				
Name	Security group rule ID	Port range	Protocol	Destination
–	sgr-05aa2c762c936bedf	5432	TCP	sg-029a4c6e001
–	sgr-02cacd496c34e4986	All	All	0.0.0.0/0

Abbildung 7: AWS Security Group Inbound-Regeln des App-Servers (Ports 80, 3000, 22, 8081, 8080).

i-067e074f5427b0bd9 (sonarqube-server)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ Instance summary [Info](#)

Instance ID i-067e074f5427b0bd9	Public IPv4 address 54.211.68.55 open address	Private IPv4 addresses 172.31.40.28
IPv6 address -	Instance state Running	Public DNS ec2-54-211-68-55.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-40-28.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-40-28.ec2.internal	
Answer private resource DNS name IPv4 (A)	Instance type t3.medium	Elastic IP addresses -
Auto-assigned IP address 54.211.68.55 [Public IP]	VPC ID vpc-025176d4c1946c7a0	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
IAM role -	Subnet ID subnet-0430b5e927e08fa39	Auto Scaling Group name -
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:223735295905:instance/i-067e074f5427b0bd9	Managed false

Abbildung 8: AWS EC2 Instanz-Details des Sonar/Jenkins-Servers.

i-067e074f5427b0bd9 (sonarqube-server)

Details | Status and alarms | Monitoring | **Security** | Networking | Storage | Tags

▼ Security details

IAM role -	Owner ID 223735295905	Launch time Tue Jul 07 2026 01:49:46 GMT+0200 (Central European Summer Time)
Security groups sg-05b8ad7242604db5e (sonarqube-sg)		

▼ Inbound rules

Q Filter rules < 1 >

Name	Security group rule ID	Port range	Protocol	Source
-	sgr-0c0713746c6c3e535	9000	TCP	0.0.0.0/0
-	sgr-0a1628c8af8944718	22	TCP	85.127.92.38/32
-	sgr-062afb76c1e10fa93	8080	TCP	85.127.92.38/32

▼ Outbound rules

Q Filter rules < 1 >

Name	Security group rule ID	Port range	Protocol	Destination
-	sgr-04ad77731f2f1a995	All	All	0.0.0.0/0

Abbildung 9: AWS Security Group Inbound-Regeln des Sonar/Jenkins-Servers (Ports 22, 8080, 9000).

Abbildung 10: AWS RDS Connectivity and Security Uebersicht.

Abbildung 11: AWS RDS Verbindungsdetails (Connection Steps, Zugangsdaten).

Connected compute resources (1) Info Connections to compute resources that were created automatically by RDS are shown here. Connections to compute resources that were created manually aren't shown.					
Filter by compute resources					
Resource identifier ↗	Resource type ▼	Availability Zone ▼	VPC security group ↗	Compute resource security group ↗	Connected proxy ↗
i-0f563fa8b4ee72f5	EC2 instance	us-east-1c	rds-ec2-1	ec2-rds-1	-

Security group rules (5) Filter by Security group rules		
Security group ▲	Type ▼	Rule
rds-ec2-1 (sg-029a4c6e011461f0)	EC2 Security Group - Inbound	sg-02415f6545b0f13b9
secret-notes-rds-sg (sg-0c8c3b0664bb93428)	CIDR/IP - Inbound	85.127.92.38/32
secret-notes-rds-sg (sg-0c8c3b0664bb93428)	CIDR/IP - Outbound	0.0.0.0/0

Replication (1) Filter by Replication					
DB identifier ▲	Role ▼	Region & AZ ▼	Replication source ▼	Replication state ▼	Lag ▼
secret-notes-db	Instance	us-east-1c	-	-	-

Abbildung 12: AWS RDS Connected Compute Resources, Security Group Rules und Replication.

7 Docker Hub

Docker Hub dient als Container Registry. Die GitHub-Actions-Pipelines bauen Images für Frontend und Backend und pushen diese mit einem commit-spezifischen Tag sowie latest. Der App-Server zieht diese Images beim Deploy und startet die entsprechenden Green-Container neu.

Bereich	Umsetzung im Projekt
DockerHub Username	if24b158
Backend Image	if24b158/secret-notes-backend:<commit-sha> und :latest
Frontend Image	if24b158/secret-notes-frontend:<commit-sha> und :latest
Delivery Stage	Authentifiziert sich mit DockerHub Token und pusht die Images.

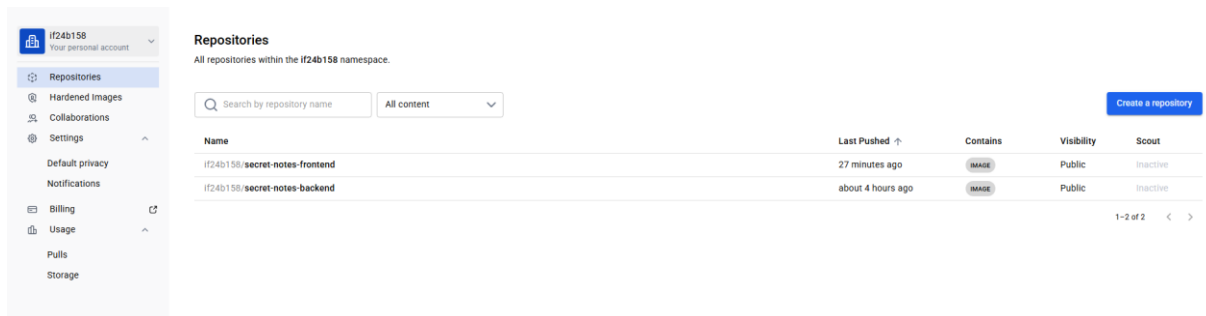


Abbildung 13: Docker Hub Repository-Uebersicht (secret-notes-frontend, secret-notes-backend).

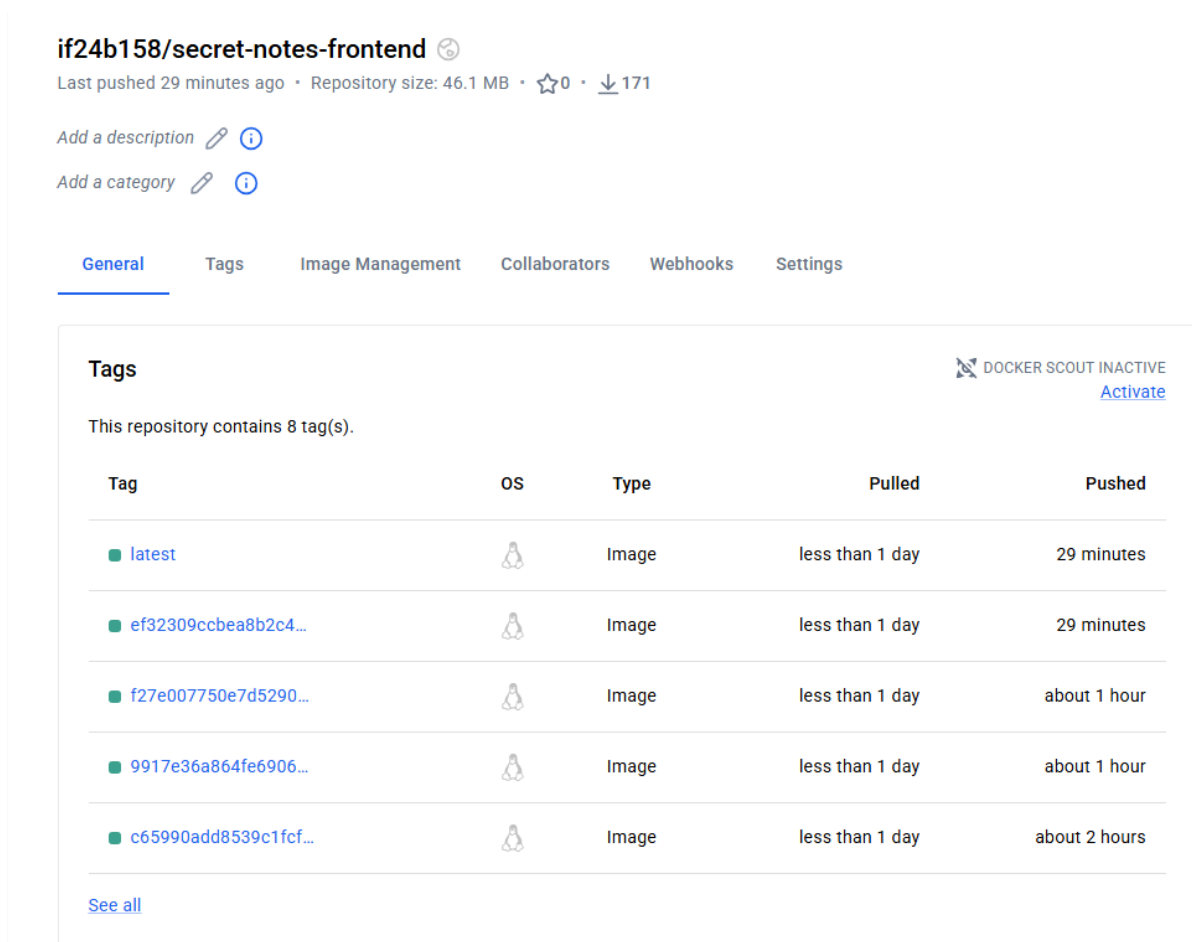


Abbildung 14: Docker Hub Frontend-Image - Tags und Push-Historie.

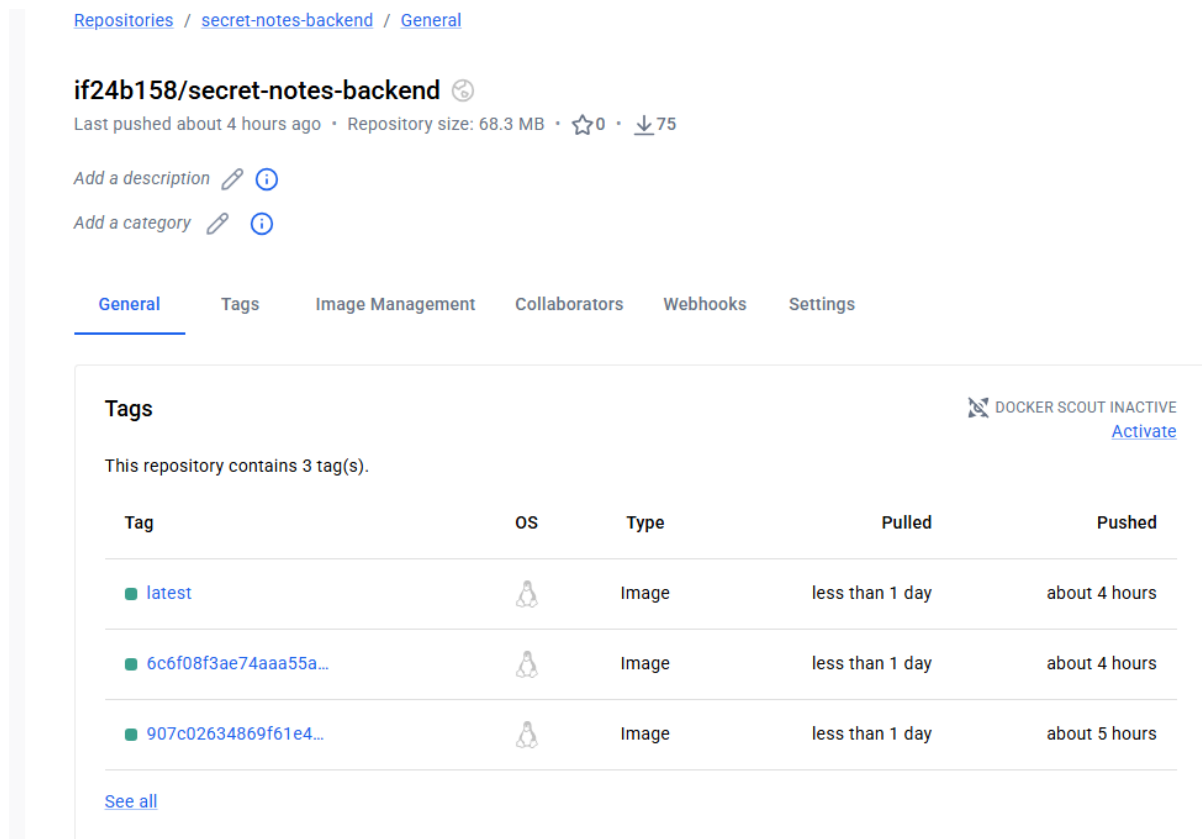


Abbildung 15: Docker Hub Backend-Image - Tags und Push-Historie.

8 CI/CD-Server: GitHub Actions und Jenkins

Die cloud-hosted CI/CD-Umgebung wurde mit GitHub Actions umgesetzt. Für Frontend und Backend existiert jeweils ein eigener Workflow. Zusätzlich wurde Jenkins als self-hosted CI auf einer AWS-EC2-Instanz eingerichtet. In Jenkins wurden Credentials für Docker Hub, SonarQube, Snyk, Slack, PostHog und den AWS-EC2-SSH-Key angelegt.

Bereich	Umsetzung im Projekt
GitHub Actions	Cloud-hosted CI/CD, führt die komplette Pipeline auf deploy/production aus.

Jenkins	Self-hosted CI auf AWS EC2, betrieben als Multibranch-Pipeline-Jobs (secret-notes-backend, secret-notes-frontend). Jenkins erkennt die Branches main und deploy/production automatisch anhand des jeweiligen Jenkinsfile und führt dieselbe Branch-Logik wie GitHub Actions aus: main = Lint/Test/Build, deploy/production = alle Stages inkl. Deliver, Deploy und E2E/Performance/Blue-Green-Switch. Beide Jobs liefen für beide Branches vollständig erfolgreich durch.
Credentials	dockerhub-credentials, sonarqube-token, snyk-token, slack-webhook-url, posthog-public-key, aws-ec2-ssh-key.
Benachrichtigung	Bei Fehlern wird der Notify-on-Failure-Job vorbereitet; Slack Webhook ist als Secret/Credential hinterlegt.
Jenkins Tool-Setup	Das offizielle jenkins/jenkins-Basis-Image enthält weder Node.js, noch den SonarQube Scanner, noch die Docker-CLI. Node.js wurde über das NodeJS-Plugin (Tool 'Node22'), der SonarQube Scanner über das SonarQube-Scanner-Plugin (Tool 'SonarScanner') als Jenkins-Tools eingebunden. Die Docker-CLI wurde direkt im laufenden Jenkins-Container installiert, da der Docker-Socket des Hosts zwar gemountet war, dem jenkins-User aber die passende Gruppenmitgliedschaft fehlte.

Jenkins Multibranch vs. einfache Pipeline	Ein zunächst angelegter einfacher 'Pipeline'-Job setzte env.BRANCH_NAME nicht, wodurch when { branch 'deploy/production' } immer false war und Deliver/Deploy/E2E stets übersprungen wurden, obwohl die Stages als 'erfolgreich' angezeigt wurden. Die Umstellung auf 'Multibranch Pipeline' behebt das, da Jenkins dabei pro Branch einen eigenen Sub-Job mit korrekt gesetztem BRANCH_NAME erzeugt.
---	---

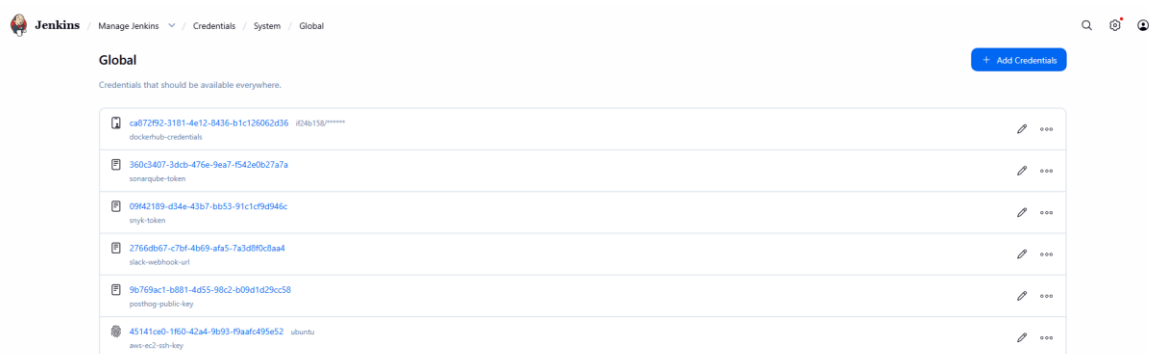


Abbildung 16: Jenkins Credentials fuer DockerHub, SonarQube, Snyk, Slack, PostHog und AWS SSH.

9 Pipeline-Stages

Die Pipeline ist für Frontend und Backend gleich aufgebaut. Auf main werden Linting, Unit Tests und Build ausgeführt. Auf deploy/production laufen alle Stages: Linting, Unit Testing, Build, Deliver, Deploy, E2E/Performance und Blue/Green Switch.

Bereich	Umsetzung im Projekt
Linting	SonarQube Scan und Snyk Security Scan.
Unit Testing	Jest/Vitest Tests mit Coverage-Artefakten.
Build	Docker Build des jeweiligen Images.
Deliver	Push der Docker Images in Docker Hub.
Deploy	SSH auf AWS App-Server, Deploy in Green-Umgebung.

E2E/Performance	Playwright Tests und k6 Performance Tests gegen Green/Staging.
Blue/Green Switch	Bei erfolgreichen Tests wird der nginx-Router auf die neue Farbe umgestellt.

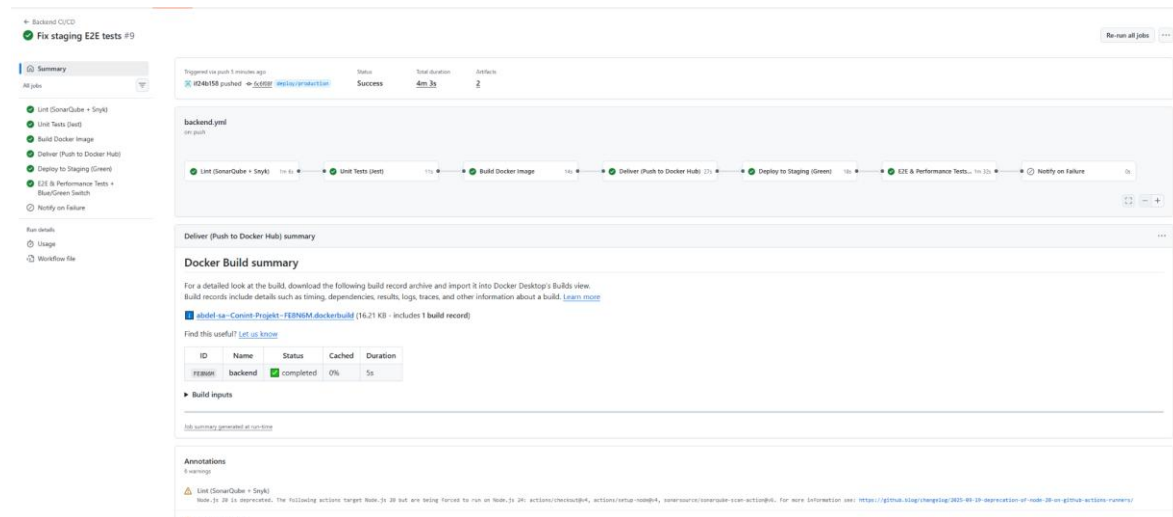


Abbildung 17: Erfolgreiche Backend-Pipeline auf deploy/production (GitHub Actions, alle Stages gruen).

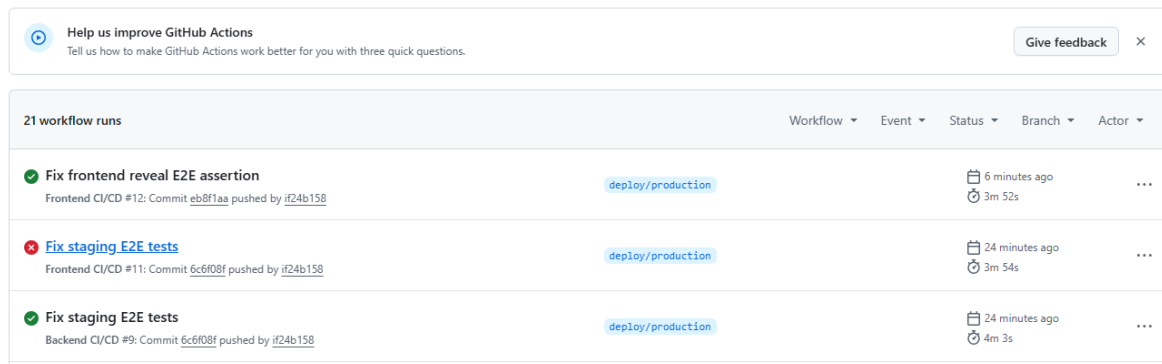


Abbildung 18: Erfolgreiche Frontend-Pipeline auf deploy/production - Workflow-Uebersicht.

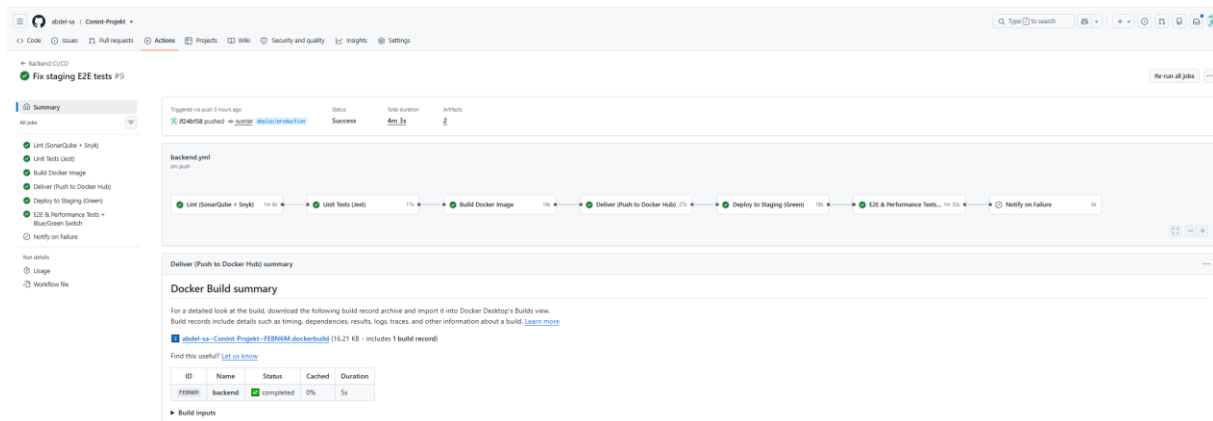


Abbildung 19: Erfolgreiche Frontend-Pipeline auf deploy/production - Detailansicht der Stages.

10 Tests und Testabdeckung

Die Unit- und Component-Tests wurden für Frontend und Backend automatisiert. Das Frontend verwendet Vitest mit Coverage, das Backend verwendet Jest. Zusätzlich laufen E2E-Tests mit Playwright gegen die Staging/Green-Umgebung. Performance-Tests werden mit k6 ausgeführt.

Bereich	Umsetzung im Projekt
Backend Tests	Jest-Tests für API- und Verschlüsselungslogik; E2E Tests gegen /api/notes und /api/health.
Frontend Tests	Vitest/Component Tests für CreateNote, RevealNote, NotesList, API und Feature Flags.
E2E Tests	Playwright erstellt eine Notiz, wählt sie aus, gibt die Passphrase ein und prüft den entschlüsselten Text.
Performance Tests	k6-Skripte prüfen Backend/Frontend-Staging-Endpunkte.
Abdeckung	Backend: 14 Jest-Tests bestanden, 100% Statement/Funktion/Zeilen-Coverage, 90% Branch-Coverage. Frontend: 19 Vitest/Component-Tests bestanden (5 Testdateien). E2E: 3 Playwright-Tests gegen die Staging/Green-Umgebung erfolgreich. Damit werden die geforderten mindestens 10 aussagekräftigen Tests je Projekt deutlich übertroffen.


```

v4.1.10 C:/Users/yoshi/Conint-Projekt/frontend
Coverage enabled with v8

✓ src/featureFlags.test.js (3 tests) 6ms
✓ src/api.test.js (6 tests) 14ms
✓ src/components/NotesList.test.jsx (2 tests) 262ms
✓ src/components/RevealNote.test.jsx (3 tests) 728ms
  ✓ reveals the plaintext content with the correct passphrase 441ms
✓ src/components/CreateNote.test.jsx (2 tests) 1016ms
  ✓ shows a success message after creating a note 767ms

Test Files 5 passed (5)
Tests 16 passed (16)
Start at 20:08:01
Duration 3.66s (transform 295ms, setup 1.11s, import 901ms, tests 2.03s, environment 8.87s)

% Coverage report from v8
-----
File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 86.36 | 67.85 | 88.88 | 87.3 |
src | 74.19 | 55 | 83.33 | 76.66 |
  api.js | 85.71 | 78.57 | 100 | 85.71 | 8,28,47
  featureFlags.js | 50 | 0 | 50 | 55.55 | 7-13
src/components | 97.14 | 100 | 91.66 | 96.96 |
  CreateNote.jsx | 100 | 100 | 100 | 100 |
  NotesList.jsx | 90 | 100 | 80 | 87.5 | 22
  RevealNote.jsx | 100 | 100 | 100 | 100 |
-----

```

Abbildung 20: Lokale Frontend-Testausgabe mit bestandenen Tests und Coverage.

11 SonarQube

SonarQube wurde self-hosted auf AWS bereitgestellt und von GitHub Actions aus angesprochen. Die Workflows übergeben SONAR_HOST_URL und SONAR_TOKEN über GitHub Secrets. Für Backend und Frontend wurden eigene Project Keys verwendet: secret-notes-backend und secret-notes-frontend.

Bereich	Umsetzung im Projekt
Server	Self-hosted SonarQube auf AWS EC2, erreichbar über Port 9000.
Backend Project Key	secret-notes-backend
Frontend Project Key	secret-notes-frontend
Ziel	Statische Analyse, Code Smells, Security Hotspots, Duplications und technische Qualitätsmetriken.
Pipeline-Integration	sonarsource/sonarqube-scan-action@v6 in beiden Workflows.

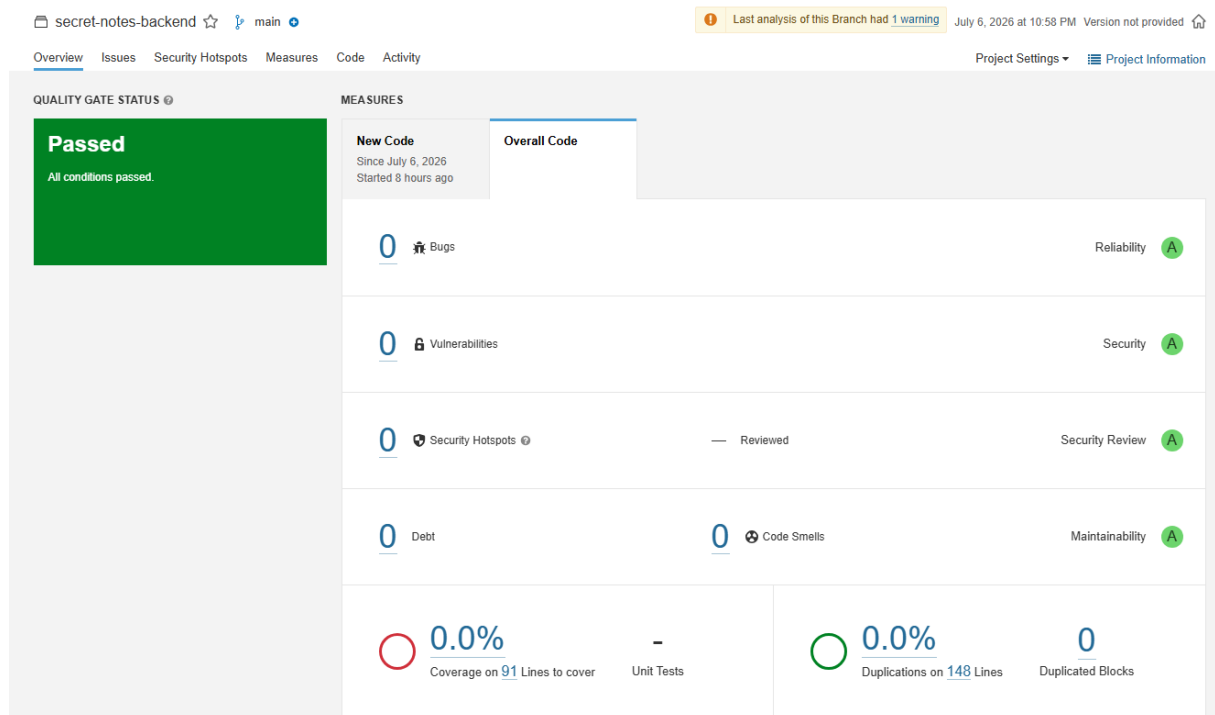


Abbildung 21: SonarQube Quality Gate Backend (secret-notes-backend) - Passed.

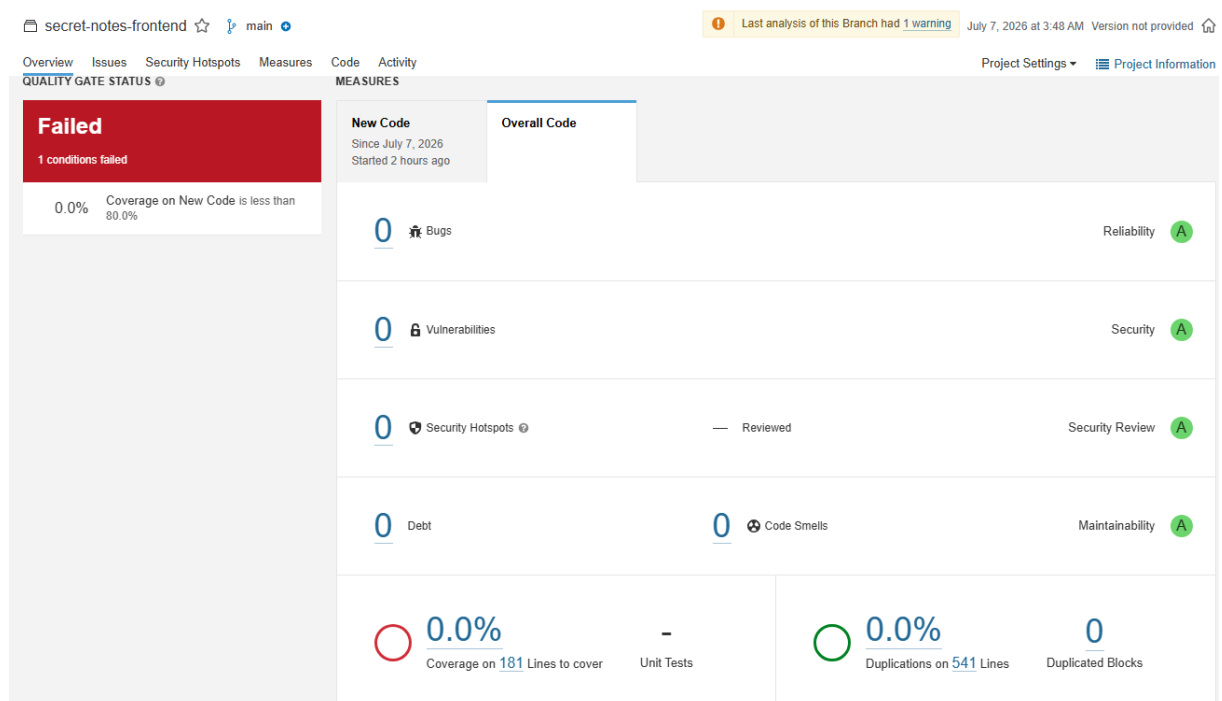


Abbildung 22: SonarQube Quality Gate Frontend (secret-notes-frontend).

12 Snyk Security Scan

Snyk wurde in der Linting-Stage integriert. Die Scans prüfen die npm-Abhängigkeiten und teilweise Container-/Dockerfile-Ziele. Im Projekt wurden gefundene High-Severity Findings analysiert. Für das Backend wurde Fastify aktualisiert, wodurch die Pipeline danach erfolgreich durchlief. Container- bzw. Base-Image-Findings werden als technische Schulden dokumentiert und können durch Aktualisierung der Basis-Images weiter reduziert werden.

Bereich	Umsetzung im Projekt
Snyk Token	Als GitHub Secret und Jenkins Credential hinterlegt.
Backend Scan	npm-Scan gegen backend/package.json (5 High, 2 Medium Findings) sowie Dockerfile-Scan gegen backend/Dockerfile (keine Findings, Base Image node:22-alpine ist sauber).
Frontend Scan	npm-Scan gegen frontend/package.json (keine Findings) sowie Dockerfile-Scan gegen frontend/Dockerfile mit Base Image nginx:1.27-alpine (3 Critical, 20 High, 10 Medium, 47 Low - insgesamt 73 Findings).
Interpretation	Der Grossteil der Findings stammt nicht aus eigenem Code, sondern aus dem Basis-Image nginx:1.27-alpine des Frontend-Dockerfiles. Snyk empfiehlt direkt im Dashboard ein minor Upgrade auf nginx:1.31.2-alpine, wodurch sich die Findings von 73 auf 13 reduzieren würden. Die 5 High-Findings im Backend stammen aus npm-Abhängigkeiten und werden durch gezielte Package-Updates behoben.
Fix-Beispiel	Snyk hat automatisch einen Pull Request '[Snyk] Security upgrade nginx from 1.27-alpine to 1.31.2-alpine' erstellt, der 4 Vulnerabilities behebt (u.a. CVE-2026-31789, OpenSSL- und libxml2-Findings). Der PR ändert nur die Base-Image-Zeile im frontend/Dockerfile und kann direkt gemerged werden.

Gescannte Targets	Insgesamt 4 Targets in Snyk importiert: frontend/Dockerfile, backend/package.json, backend/Dockerfile, frontend/package.json.
-------------------	---

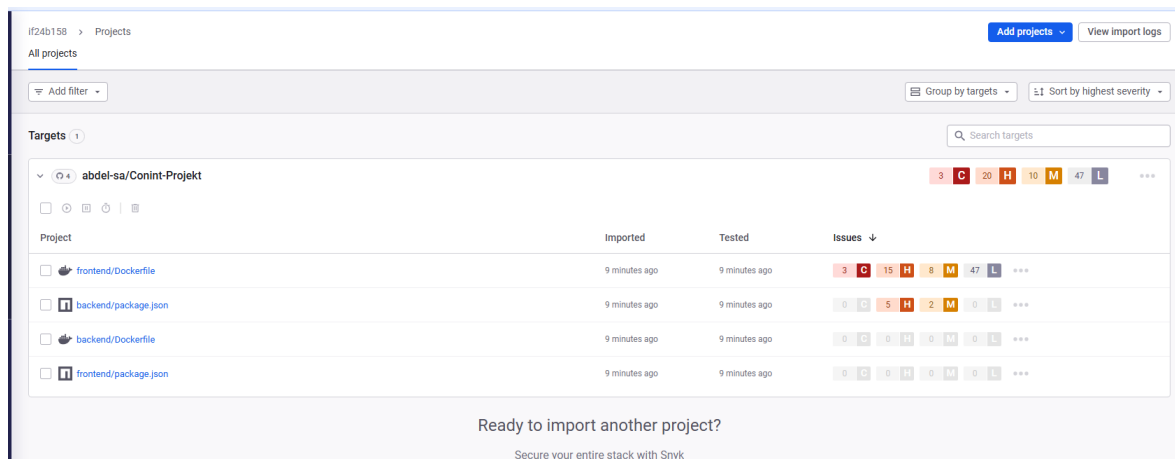


Abbildung 23: Snyk-Projektuebersicht mit analysierten Targets und Findings.

13 PostHog Feature Toggle und A/B-Test

PostHog wurde für ein Feature Flag eingerichtet. Das Flag secret-notes-ui-variant-b ist aktiviert und auf einen 50%-Rollout konfiguriert. Damit kann eine UI-Variante oder ein alternativer Flow für einen Teil der Benutzer aktiviert werden, ohne die Anwendung neu zu deployen.

Bereich	Umsetzung im Projekt
Feature Flag	secret-notes-ui-variant-b
Rollout	50% der Benutzer im Condition Set.
Nutzung im Frontend	Frontend liest PostHog Public Key und Host aus Build-/Runtime-Konfiguration.
Zweck	A/B-Test für UI-Variante bzw. Experimente mit Nutzererfahrung.

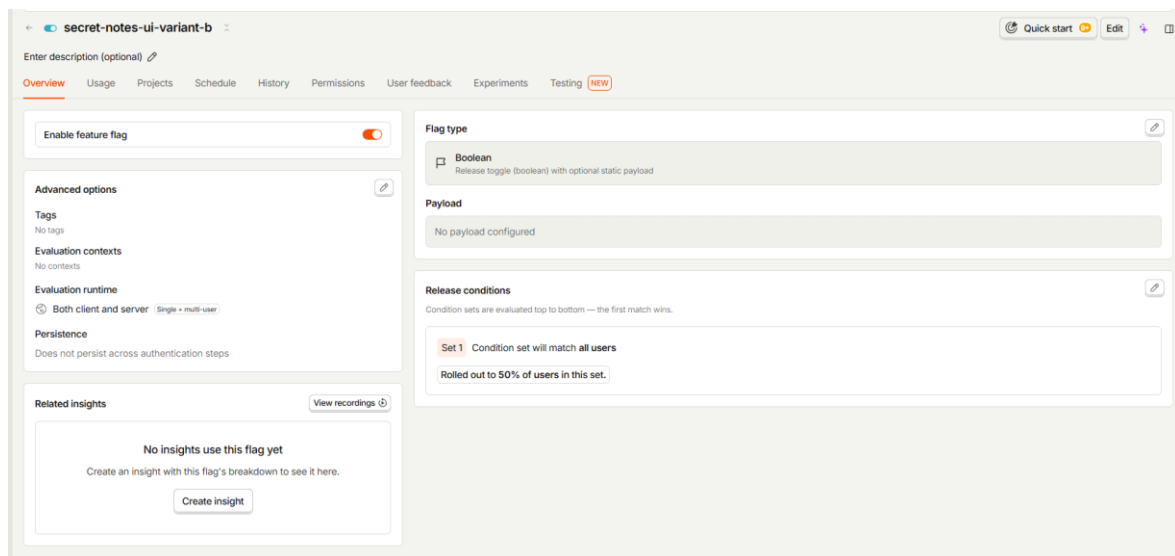


Abbildung 24: PostHog Feature Flag secret-notes-ui-variant-b mit 50%-Rollout.

14 Blue/Green Deployment

Das Blue/Green Deployment wird über Docker Compose und einen nginx-Router umgesetzt. Beide Umgebungen laufen parallel: backend-blue/frontend-blue und backend-green/frontend-green. Port 8081 dient als Green Preview. Der Live-Traffic auf Port 80 wird über eine active-color.conf auf die aktive Farbe geleitet.

Bereich	Umsetzung im Projekt
Blue Environment	Aktuelle oder vorherige Version, erreichbar intern über backend-blue und frontend-blue.
Green Environment	Neue Staging-Version, erreichbar über Port 8081 für Tests.
Router	nginx-Router inkludiert active-color.conf und leitet /api/* an das aktive Backend.
Switch	switch-blue-green.sh kopiert active-color-green.conf oder active-color-blue.conf und lädt nginx neu.
Absicherung	Erst nach erfolgreichen Playwright- und k6-Tests wird der Switch ausgeführt.

Nachweis (docker ps -a auf dem App-Server, 07.07.2026): Die fünf erwarteten Container liefen parallel - deploy-frontend-blue-1, deploy-frontend-green-1, deploy-backend-blue-1, deploy-backend-green-1 und deploy-router-1 (nginx, Port 80 und 8081 gemappt). Siehe Abbildung 25 für den zugehörigen Screenshot.

```
ubuntu@ip-172-31-3-27:/opt/secret-notes/deploy$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
42bee001b8ef	if24b158/secret-notes-frontend:70ecf1ab0565ff59f77fff16ba2c0c92aa6cc371	deploy-frontend-green-1	"/docker-entrypoint..."	11 minutes ago	Up 11 minutes	80/tcp
7d3fcc6a9e0	if24b158/secret-notes-backend:latest	deploy-backend-green-1	"/docker-entrypoint.s..."	2 hours ago	Up 2 hours	3000/tcp
fad1c807628	if24b158/secret-notes-backend:latest	deploy-backend-blue-1	"/docker-entrypoint.s..."	10 hours ago	Up 2 hours	3000/tcp
8e1b87cc78c4	nginx:1.27-alpine	deploy-router-1	"/docker-entrypoint..."	10 hours ago	Up 2 hours	0.0.0.0:80->80/tcp, [::]:80->80/tcp
58ae31fa9dec	if24b158/secret-notes-frontend:latest	deploy-frontend-blue-1	"/docker-entrypoint..."	10 hours ago	Up 2 hours	80/tcp

Abbildung 25: Docker ps-Ausgabe auf dem App-Server - Blue/Green-Container laufen parallel.

15 Fehlerbehandlung und Benachrichtigung

Fehlgeschlagene Stages stoppen die nachfolgenden Pipeline-Schritte. Für Fehlerfälle wurde ein Notify-on-Failure-Job vorbereitet, der über einen Slack Webhook informieren kann. Während der Umsetzung traten typische CI/CD-Probleme auf, z.B. nicht erreichbarer SonarQube-Port, fehlender sonar.projectKey, falsche YAML-Einrückung, Snyk-Findings, Lockfile-Inkonsistenzen und fehlerhafte E2E-Selektoren. Diese Probleme wurden schrittweise behoben und dokumentiert.

16 Benutzerfunktionen und User Stories

Die Anwendung erfüllt die drei Hauptfunktionen aus der Angabe: Secret Note erstellen, Secret Note lesen und UI-Variante per Feature Toggle steuern.

Bereich	Umsetzung im Projekt
User Story 1: Create Secret Note	Benutzer gibt Text und Passphrase ein. Das Backend verschlüsselt den Inhalt und speichert die Notiz. Die UI bestätigt die sichere Erstellung.
User Story 2: Read Secret Note	Benutzer wählt eine Note-ID, gibt die Passphrase ein und erhält nur bei korrekter Passphrase den Klartext.
User Story 3: Feature Toggle	PostHog steuert eine UI-Variante bzw. A/B-Test-Gruppe ohne erneutes Deployment.

17 Setup- und Bedienungsanleitung

17.1 Lokale Entwicklung

1. Repository klonen und in das Projektverzeichnis wechseln.
2. Backend-Abhängigkeiten installieren: `cd backend && npm ci`.
3. Frontend-Abhängigkeiten installieren: `cd frontend && npm ci`.
4. Unit Tests ausführen: `npm test` bzw. `npm test -- --coverage`.
5. Docker Images lokal bauen oder über GitHub Actions bauen lassen.

17.2 AWS-Betrieb

1. RDS PostgreSQL muss erreichbar sein und die Tabelle notes enthalten.
2. App-Server per SSH öffnen.
3. In /opt/secret-notes/deploy wechseln.
4. Docker Compose starten: `sudo docker compose --env-file .env -f docker-compose.blue-green.yml up -d`.
5. Health prüfen: `curl http://localhost/api/health` und `curl http://localhost:8081/api/health`.

17.3 Bedienung der Anwendung

1. Live URL öffnen.
2. Note content und Passphrase eingeben.
3. Create note klicken.
4. Erstellte Note-ID in der Liste auswählen.
5. Dieselbe Passphrase eingeben und Reveal klicken.
6. Der Klartext wird nur bei korrekter Passphrase angezeigt.

18 Probleme, Entscheidungen und Lösungen

Bereich	Umsetzung im Projekt
SonarQube Timeout	Port 9000 war nicht für GitHub Actions erreichbar. Lösung: Security Group für 9000 öffnen.
sonar.projectKey fehlt	Workflows erhielten args mit SONAR_PROJECT_KEY_BACKEND und SONAR_PROJECT_KEY_FRONTEND.
YAML-Fehler	Einrückung von uses/env/with korrigiert.
Snyk High Findings	Fastify wurde aktualisiert; Snyk-Pfade wurden explizit auf backend/package.json und frontend/package.json gesetzt.
Frontend package-lock	package-lock.json wurde neu erzeugt und mit package.json synchronisiert.
E2E: falscher Endpoint	Backend E2E nutzt /api/health statt /health.
E2E: doppelte Passphrase-Labels	Frontend E2E nutzt form().first() und form().nth(1), um Create- und Reveal-Passphrase eindeutig zu unterscheiden.
Frontend Failed to fetch	STAGING_API_URL wurde auf /api gesetzt, damit der Browser relativ zur aktuellen Origin auf die API zugreift.
Windows-Zeileneenden	.gitattributes erzwingt LF für Shell-Skripte im deploy-Verzeichnis.

Jenkins: npm not found	Der offizielle jenkins/jenkins-Container bringt kein Node.js mit, wodurch 'npm ci' im Lint-Stage sofort fehlschlug. Lösung: NodeJS-Plugin installiert, Tool 'Node22' unter Global Tool Configuration angelegt und per tools { nodejs 'Node22' } im Jenkinsfile eingebunden.
Jenkins: sonar-scanner not found	Gleiches Muster wie bei npm: der SonarQube-Scanner ist im Basis-Image nicht enthalten. Lösung: SonarQube-Scanner-Plugin installiert, Tool 'SonarScanner' konfiguriert und über withEnv(["PATH+SONAR=\${tool 'SonarScanner'}/bin"]) in den PATH der Lint-Stage aufgenommen.
Jenkins: docker not found / permission denied	Jenkins läuft selbst als Docker-Container. Der Docker-Socket des Hosts war zwar nach /var/run/docker.sock gemountet, es fehlte aber sowohl die Docker-CLI im Container als auch die passende Gruppenmitgliedschaft des jenkins-Users für den Socket. Lösung: Docker-CLI-Binary im Container installiert, jenkins-User einer Gruppe mit der GID der Host-Docker-Gruppe hinzugefügt und der Container einmalig neu gestartet.
Jenkins: sshagent-Schritt nicht gefunden	Der Deploy-Stage nutzt sshagent(['aws-ec2-ssh-key']) für den SSH-Zugriff auf den App-Server; das dafür nötige SSH-Agent-Plugin war nicht installiert. Lösung: Plugin installiert, Jenkins neu gestartet.
Jenkins: k6 not found	k6 war im Jenkins-Container für den Performance-Test-Schritt nicht vorhanden. Lösung: aktuelles k6-Release-Binary direkt in den Container geladen (kein offizielles Jenkins-Tool verfügbar).

Jenkins: Deliver/Deploy/E2E wurden trotz deploy/production übersprungen	Bei einem einfachen 'Pipeline script from SCM'-Job wird env.BRANCH_NAME nicht gesetzt, wodurch when { branch 'deploy/production' } immer false auswertete und die Stages in Millisekunden als 'erfolgreich' übersprungen wurden, ohne etwas auszuführen. Lösung: Umstellung auf 'Multibranch Pipeline', wodurch Jenkins pro erkanntem Branch einen eigenen Sub-Job mit korrektem BRANCH_NAME erzeugt.
PostHog Feature Flag griff im deployten Frontend nicht	Zwei überlagerte Ursachen: (1) VITE_POSTHOG_HOST wurde beim Docker-Build weder in GitHub Actions noch im Jenkinsfile als Build-Arg übergeben, wodurch das Frontend immer den US-Host statt des tatsächlich genutzten EU-Hosts ansprach (401 Unauthorized). (2) isEnabled() wurde direkt nach posthog.init() aufgerufen, bevor die Flags asynchron vom Server geladen waren (Race Condition). Lösung: VITE_POSTHOG_HOST als zusätzliches Build-Arg ergänzt und die Flag-Auswertung über den posthog.onFeatureFlags()-Callback verzögert.
main-Branch weicht von deploy/production ab	Da Bugfixes zunächst nur direkt auf deploy/production committet wurden, schlug die main-Pipeline in Jenkins mit denselben, bereits gelösten Fehlern (z.B. npm not found) erneut fehl, weil main das alte Jenkinsfile verwendete. Lösung: deploy/production wurde nach main gemergt, damit beide Branches denselben, funktionierenden Stand haben.

19 Logging und Monitoring (Outlook)

Aktuell werden Pipeline-Logs in GitHub Actions und Jenkins sowie Container-Logs über Docker bereitgestellt. Für einen produktionsnahen Ausbau wären CloudWatch Logs,

strukturierte JSON-Logs im Backend, ein zentrales Dashboard und Alerting für Fehlerquoten und Response Times sinnvoll. PostHog kann zusätzlich für Produkt- und Nutzungsmetriken verwendet werden.

20 Fazit

Das Projekt Secret Notes wurde als vollständige CI/CD-Uebung umgesetzt. Die Anwendung läuft containerisiert auf AWS, speichert Notizen verschlüsselt in RDS und wird über GitHub Actions automatisiert gebaut, getestet, ausgeliefert und deployed. Die deploy/production-Pipelines für Frontend und Backend laufen erfolgreich durch und zeigen die geforderten Stages inklusive Docker Hub Delivery, Deploy, E2E/Performance und Blue/Green Switch. Offene produktionsnahe Erweiterungen betreffen vor allem dauerhaftes Monitoring, weitere Härtung der Infrastruktur und vollständige Jenkins-Pipeline-Nachweise.

21 Literaturverzeichnis

- [1] AWS Documentation: Amazon EC2, Amazon RDS, Security Groups.
- [2] Docker Documentation: Dockerfile, Docker Compose, Docker Hub.
- [3] GitHub Documentation: GitHub Actions Workflows, Repository Secrets.
- [4] SonarSource Documentation: SonarQube Scanner for GitHub Actions.
- [5] Snyk Documentation: npm Dependency Scanning and Container Scanning.
- [6] PostHog Documentation: Feature Flags and Experiments.
- [7] Playwright Documentation: End-to-End Testing.
- [8] Grafana k6 Documentation: Performance Testing.

22 Abbildungsverzeichnis

- Abbildung 1: Secret Notes Frontend mit erstellter und korrekt entschlüsselter Notiz.
- Abbildung 2: Datenbanknachweis: gespeicherte Notiz nur als ciphertext, iv, salt und auth_tag.
- Abbildung 3: Backend Dockerfile - Node.js-Basis-Image, Installation der Dependencies und Start des Fastify-Servers.
- Abbildung 4: docker-compose.blue-green.yml - Definition der Blue/Green-Services (Backend, Frontend, Router).
- Abbildung 5: Frontend Dockerfile - Multi-Stage-Build mit Vite und Auslieferung über nginx.
- Abbildung 6: GitHub Repository Secrets für Deployment, SonarQube, Snyk, Slack und PostHog.

Abbildung 7: AWS Security Group Inbound-Regeln des App-Servers (Ports 80, 3000, 22, 8081, 8080).

Abbildung 8: AWS EC2 Instanz-Details des Sonar/Jenkins-Servers.

Abbildung 9: AWS Security Group Inbound-Regeln des Sonar/Jenkins-Servers (Ports 22, 8080, 9000).

Abbildung 10: AWS RDS Connectivity and Security Übersicht.

Abbildung 11: AWS RDS Verbindungsdetails (Connection Steps, Zugangsdaten).

Abbildung 12: AWS RDS Connected Compute Resources, Security Group Rules und Replication.

Abbildung 13: Docker Hub Repository-Übersicht (secret-notes-frontend, secret-notes-backend).

Abbildung 14: Docker Hub Frontend-Image - Tags und Push-Historie.

Abbildung 15: Docker Hub Backend-Image - Tags und Push-Historie.

Abbildung 16: Jenkins Credentials für DockerHub, SonarQube, Snyk, Slack, PostHog und AWS SSH.

Abbildung 17: Erfolgreiche Backend-Pipeline auf deploy/production (GitHub Actions, alle Stages grün).

Abbildung 18: Erfolgreiche Frontend-Pipeline auf deploy/production - Workflow-Übersicht.

Abbildung 19: Erfolgreiche Frontend-Pipeline auf deploy/production - Detailansicht der Stages.

Abbildung 20: Lokale Frontend-Testausgabe mit bestandenen Tests und Coverage.

Abbildung 21: SonarQube Quality Gate Backend (secret-notes-backend) - Passed.

Abbildung 22: SonarQube Quality Gate Frontend (secret-notes-frontend).

Abbildung 23: Snyk-Projektübersicht mit analysierten Targets und Findings.

Abbildung 24: PostHog Feature Flag secret-notes-ui-variant-b mit 50%-Rollout.

Abbildung 25: Docker ps-Ausgabe auf dem App-Server - Blue/Green-Container laufen parallel.

23 Abkuerzungsverzeichnis

Abkuerzung	Bedeutung
API	Application Programming Interface
CI	Continuous Integration
CD	Continuous Deployment / Continuous Delivery
EC2	Elastic Compute Cloud
E2E	End-to-End
RDS	Relational Database Service

UI	User Interface
SSH	Secure Shell
URL	Uniform Resource Locator

24 Dokumentationstabelle KI-basierte Hilfsmittel

KI-basiertes Hilfsmittel	Verwendungszweck	Prompt / Quelle / Bereich
ChatGPT	Technische Unterstützung bei CI/CD-Fehlersuche, Formulierung der Dokumentation, Strukturierung nach Projektangabe.	Unterstützung bei technischen Rückfragen, Fehlersuche
GitHub Actions Logs	Fehleranalyse der Pipeline-Stages.	Pipeline-Ausgaben aus Backend/Frontend Workflows.
AWS/GitHub/Snyk/PostHog/Jenkins Oberflächen	Screenshots und Konfigurationsnachweise.	Screenshots aus der Projektumgebung.